

Simulação Discreta de Filas para um Roteador IoT com Buffer Inteligente e Descarte Seletivo

Matheus Ferreira Silva e Eduardo Costa Resende

Instituto Nacional de Telecomunicações (Inatel), Santa Rita do Sapucaí, MG, Brasil

Abstract—Este trabalho apresenta um estudo de simulação discreta orientada a eventos para um roteador Internet das Coisas (IoT) com capacidade limitada de armazenamento, modelado como uma fila M/M/1 com buffer inteligente e descarte seletivo. No sistema proposto, chegadas seguem um processo de Poisson com taxa λ , o serviço é exponencial com taxa μ , o buffer comporta até K pacotes em espera e cada pacote pertence a uma das classes de prioridade alta ou baixa. Quando o buffer está cheio, pacotes de baixa prioridade são descartados imediatamente, enquanto pacotes de alta prioridade podem substituir o pacote de baixa prioridade mais antigo presente na fila. O objetivo é quantificar o ganho de proteção à classe prioritária e comparar esse comportamento com filas de referência M/M/1/N/FCFS, com $N = K + 1$, M/M/1/ ∞ /FCFS, M/G/1/ ∞ /FCFS, M/M/m/ ∞ /FCFS e prioridade sem preempção. Foram executadas 30 replicações por cenário, com aquecimento de 100 s e janela medida de 900 s. No cenário base, a fila inteligente manteve probabilidade de perda total semelhante à M/M/1/N/FCFS tradicional, porém reduziu a perda da classe alta de 6,57% para 0,13%, com aumento da vazão prioritária de 56,09 para 59,89 pacotes/s. Os resultados mostram que o mecanismo atende ao objetivo de proteger tráfego crítico, ao custo de concentrar perdas na classe baixa.

Index Terms—simulação discreta, teoria de filas, Internet das Coisas, prioridade de pacotes, buffer finito, roteadores.

I. INTRODUÇÃO

Aplicações de Internet das Coisas (IoT) combinam grande número de dispositivos, buffers modestos e requisitos heterogêneos de latência e confiabilidade. Nessas condições, a forma como a fila trata sobrecarga e competição por armazenamento afeta diretamente a qualidade de serviço (QoS) percebida por fluxos críticos, como telemetria industrial, alarmes e mensagens de controle. Em sistemas IoT embarcados e de tempo real, a sobrecarga causada pelo processamento indiscriminado de pacotes pode comprometer prazos de execução, e mecanismos de diferenciação de fluxos têm sido estudados para proteger o sistema em cenários de tráfego elevado [1]. Além disso, arquiteturas recentes de baixa latência, como L4S, reforçam que atraso de enfileiramento, perda e vazão continuam sendo métricas centrais no projeto de redes modernas [2].

Em cenários com múltiplas classes, a disciplina de atendimento precisa equilibrar proteção ao tráfego urgente com uso eficiente do buffer. Domański et al. [3] propõem um mecanismo de *active queue management* para gateway IoT baseado em classificação de pacotes, no qual o descarte é concentrado sobre tráfego não prioritário para preservar pacotes críticos. De forma complementar, estudos em NB-IoT mostram que o transbordamento de filas é causa direta de perdas e atrasos, motivando técnicas de gerenciamento ativo

de filas para melhorar vazão, atraso e taxa de entrega [4]. Abdollahi et al. [5] analisam precisamente esse dilema em um sistema IoT com prioridades e buffer unitário, mostrando que políticas não preemptivas e critérios de substituição impactam simultaneamente perda, vazão e atualização da informação. O problema deste trabalho segue essa motivação, mas foi formulado como um estudo de simulação de uma fila M/M/1 com buffer inteligente e descarte seletivo, na qual pacotes de alta prioridade podem expulsar pacotes de baixa prioridade em espera quando o buffer está cheio.

O objetivo é comparar a fila proposta com modelos clássicos de referência e responder três perguntas. Primeiro, qual é o ganho efetivo de proteção à classe alta quando o buffer passa a aceitar substituição seletiva. Segundo, qual é o custo imposto à classe baixa e à perda total do sistema. Terceiro, como os indicadores mudam com a utilização ρ , com o tamanho do buffer K , com a mistura entre prioridades e com a variabilidade do serviço.

II. MODELAGEM DO SISTEMA

A. Fila principal

O sistema principal é uma fila M/M/1 com chegadas Poisson, serviço exponencial, disciplina first-come, first-served (FCFS) no servidor e buffer finito com política de descarte seletivo. A intensidade de tráfego é dada por

$$\rho = \frac{\lambda}{\mu}. \quad (1)$$

Cada chegada recebe prioridade alta com probabilidade p e prioridade baixa com probabilidade $1 - p$. O servidor atende um pacote por vez e o buffer comporta até K pacotes aguardando atendimento, de modo que a capacidade total do sistema é $N = K + 1$.

Quando há espaço disponível, qualquer pacote é aceito. Quando o buffer está cheio, a política proposta opera em três casos:

- 1) uma chegada de baixa prioridade é descartada imediatamente;
- 2) uma chegada de alta prioridade substitui o pacote de baixa prioridade mais antigo presente apenas na fila de espera;
- 3) se todos os pacotes em espera também forem de alta prioridade, a nova chegada de alta prioridade é descartada.

Essa regra preserva o pacote já em serviço e concentra a decisão de substituição somente no buffer, o que torna o mecanismo comparável a uma forma de gerenciamento seletivo de admissão.

B. Filas de comparação

Para contextualizar o desempenho da fila proposta, foram simuladas cinco referências:

- 1) M/M/1/N/FCFS, com $N = K + 1$, que possui o mesmo servidor e o mesmo buffer finito, mas bloqueia qualquer chegada quando o sistema está cheio;
- 2) M/M/1/∞/FCFS, usada como referência sem perdas por buffer;
- 3) M/G/1/∞/FCFS, com a mesma média de serviço da M/M/1 e três distribuições de serviço: exponencial, determinística e uniforme;
- 4) M/M/m/∞/FCFS, com $m = 2$ servidores paralelos;
- 5) fila com prioridade sem preempção, com duas filas lógicas por classe e escolha prioritária do próximo pacote após cada partida.

C. Variáveis de estado, entidades e eventos

As entidades do sistema são os pacotes. Cada pacote armazena identificador, classe de prioridade, instante de chegada, instante de início de serviço, duração de serviço, instante de saída e, quando aplicável, motivo de descarte.

As principais variáveis de estado são: número de pacotes no sistema $n(t)$, número de pacotes em espera $n_q(t)$, indicador de servidor ocupado $b(t)$ e composição do buffer por classe. No modelo com prioridade sem preempção, a composição é separada em duas filas lógicas. No modelo inteligente, também é necessário saber qual é o pacote de baixa prioridade mais antigo ainda em espera.

Os eventos elementares são:

- 1) *chegada*, que cria um novo pacote, escolhe sua classe, atualiza contadores e decide entre admissão, enfileiramento, descarte ou substituição;
- 2) *partida*, que remove o pacote em serviço, acumula estatísticas temporais e inicia o atendimento do próximo pacote elegível, quando houver.

III. METODOLOGIA DE SIMULAÇÃO

Foi utilizada simulação discreta orientada a eventos, implementada em Python, com calendário de eventos baseado em heap e coleta explícita de áreas de ocupação para estimativas temporais. Todas as filas foram simuladas por 1000 s, com descarte dos primeiros 100 s como aquecimento. Cada cenário foi repetido 30 vezes com sementes determinísticas derivadas de uma semente-base comum, o que permite reprodutibilidade e comparações consistentes entre cenários.

A Tabela I resume os parâmetros base. Além do cenário nominal, foram avaliadas variações de utilização, de tamanho de buffer, de mistura entre prioridades e de distribuição de serviço. Os resultados foram estimados a partir de 30 replicações, e os intervalos de confiança de 95% foram utilizados na análise e na geração dos gráficos quando aplicável.

As métricas avaliadas incluem probabilidade de perda total e por classe, probabilidade de substituição de pacotes de baixa prioridade, vazão total e por classe, utilização do servidor, ocupação média do sistema, tempo médio de espera, tempo médio no sistema e percentis do tempo no sistema. Também

TABLE I
PARÂMETROS BASE DO ESTUDO

Parâmetro	Valor
λ	200 pacotes/s
μ	250 pacotes/s
ρ	0,8
K	5 pacotes em espera
p	0,3
Servidores	1, exceto na M/M/m
Tempo de simulação	1000 s
Aquecimento	100 s
Replicações	30

foi verificada a consistência com a Lei de Little. Para a fila inteligente, duas checagens foram registradas: uma forma direta e uma forma ajustada ao fluxo efetivamente admitido, que considera pacotes substituídos no buffer.

IV. RESULTADOS E DISCUSSÃO

A. Comparação entre filas no cenário base

A Tabela II resume os principais indicadores médios do cenário base. Como esperado, as filas com buffer infinito não apresentam perda por bloqueio. A fila M/M/m com dois servidores reduz drasticamente o atraso, enquanto a fila com prioridade sem preempção desloca o custo do atendimento preferencial da classe alta para a classe baixa, sem introduzir perdas, pois o buffer é infinito.

A comparação entre a fila inteligente e a M/M/1/N/FCFS é o ponto central do estudo. A Fig. 1 mostra que a probabilidade de perda total permaneceu praticamente igual entre ambas, em torno de 6,6%. Isso indica que o mecanismo proposto não aumenta o volume global de descarte, apenas redistribui onde a perda acontece.

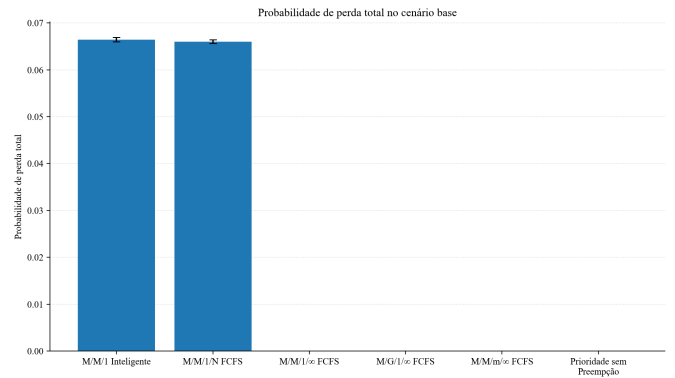


Fig. 1. Probabilidade de perda total no cenário base.

O ganho aparece quando a análise é feita por classe. Na fila FCFS finita, as probabilidades de perda das classes alta e baixa foram praticamente idênticas, 6,57% e 6,62%, respectivamente. Na fila inteligente, a perda da classe alta caiu para 0,13%, o que representa redução de aproximadamente 98%, enquanto a perda da classe baixa subiu para 9,43%. Em

TABLE II
COMPARAÇÃO ENTRE FILAS NO CENÁRIO BASE

Fila	Perda total (%)	Tempo médio no sistema (ms)	Vazão total (pacotes/s)	Utilização (%)
M/M/1 Inteligente	6,64	11,13	186,72	74,71
M/M/1/N FCFS	6,60	11,46	186,85	74,66
M/M/1 infinito FCFS	0,00	19,88	199,93	79,95
M/G/1 infinito FCFS	0,00	19,91	199,93	80,00
M/M/2 infinito FCFS	0,00	4,76	200,00	39,99
Prioridade sem preempção	0,00	19,84	199,98	79,96

paralelo, a vazão da classe alta aumentou 6,78% e a da classe baixa caiu 3,00%. A Fig. 2 sintetiza esse deslocamento de proteção.

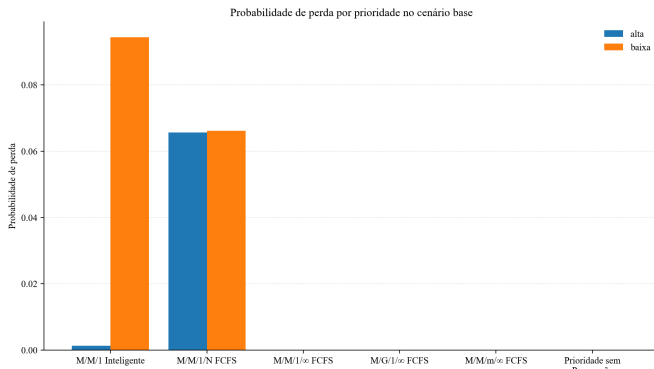


Fig. 2. Probabilidade de perda por classe no cenário base.

Outro resultado importante é que a proteção à classe alta quase não alterou seu tempo médio de espera em relação à M/M/1/N/FCFS. No cenário base, o tempo médio no sistema da classe alta ficou em 11,45 ms na fila inteligente e 11,45 ms na fila finita tradicional, enquanto a classe baixa caiu levemente para 10,98 ms devido ao descarte seletivo de parte dos pacotes que sofreriam espera mais longa. Em outras palavras, o principal efeito da política não foi reduzir atraso médio, e sim realocar perdas em favor do tráfego prioritário. Esse resultado evidencia que a política proposta atua principalmente como mecanismo de realocação de perdas, e não como aumento de capacidade. Esse comportamento é esperado em sistemas de buffer finito: sem alterar λ , μ ou K , o gerenciamento da fila muda quais pacotes são preservados, mas não remove a restrição física de armazenamento. Políticas modernas de AQM seguem lógica semelhante ao controlar a dinâmica de descarte e enfileiramento para melhorar métricas de QoS sob congestionamento [3, 4].

B. Efeito da utilização, do buffer e da variabilidade do serviço

A Fig. 3 mostra que o aumento de ρ eleva o tempo médio no sistema em todas as filas com um único servidor. Nas filas com buffer infinito, o crescimento é mais acentuado e chega a cerca de 80 ms para $\rho = 0,95$, refletindo o comportamento esperado de saturação. A fila inteligente acompanha de perto a M/M/1/N/FCFS, mantendo atraso abaixo de 13 ms mesmo

em alta carga porque a limitação de buffer converte parte da sobrecarga em descarte.

Ao variar K , o comportamento da fila inteligente permaneceu muito próximo ao da M/M/1/N/FCFS em perda total, como mostra a Fig. 4. Ainda assim, o ganho de seletividade continua presente por classe. Na fila inteligente, a perda total subiu para 17,31% quando $K = 2$, caiu para 1,84% quando $K = 20$ e, no mesmo movimento, a perda da classe alta variou de 3,47% para 0%. Isso confirma que buffers maiores reduzem bloqueio para todos, mas o mecanismo seletivo preserva a prioridade mesmo em tamanhos menores.

Na M/G/1, o valor médio do serviço foi mantido constante e apenas a distribuição foi alterada. A Fig. 5 mostra que a distribuição determinística reduziu o tempo médio no sistema em 39,6% frente ao caso exponencial, enquanto a distribuição uniforme reduziu esse tempo em 26,3%. O resultado está de acordo com a intuição de que menor variabilidade de serviço reduz flutuações da fila.

C. Influência da mistura entre prioridades e validações

Quando a fração de pacotes prioritários aumenta, a fila inteligente perde parte da folga necessária para proteger a classe alta por substituição de pacotes baixos. Com $p = 0,1$, a perda da classe alta foi praticamente nula. Com $p = 0,5$, essa perda subiu para 0,94%, e com $p = 0,8$ chegou a 3,97%. Ao mesmo tempo, a probabilidade de substituição de pacotes baixos cresceu de 0,73% para 10,80%, indicando que a política atua mais intensamente justamente quando há competição real entre classes. O aumento da fração de pacotes prioritários também mostra uma limitação da estratégia: quando muitos pacotes são de alta prioridade, há menos pacotes de baixa prioridade disponíveis para substituição, reduzindo o ganho seletivo da política. Esse efeito é análogo ao problema de *starvation* em mecanismos com prioridade. Em DualQ AQM, por exemplo, a fila de baixa latência recebe prioridade, mas essa prioridade precisa ser limitada para evitar que a fila clássica seja excessivamente prejudicada [6].

As verificações de consistência também foram satisfatórias. Para quase todas as filas, o erro relativo direto da Lei de Little ficou da ordem de 10^{-5} . Na fila inteligente, a checagem direta apresentou erro relativo de 3,05%, enquanto a versão ajustada ao fluxo admitido caiu para $4,62 \times 10^{-5}$. Esse resultado era esperado porque a política de substituição cria pacotes que ocupam o sistema por algum tempo, mas deixam de contribuir

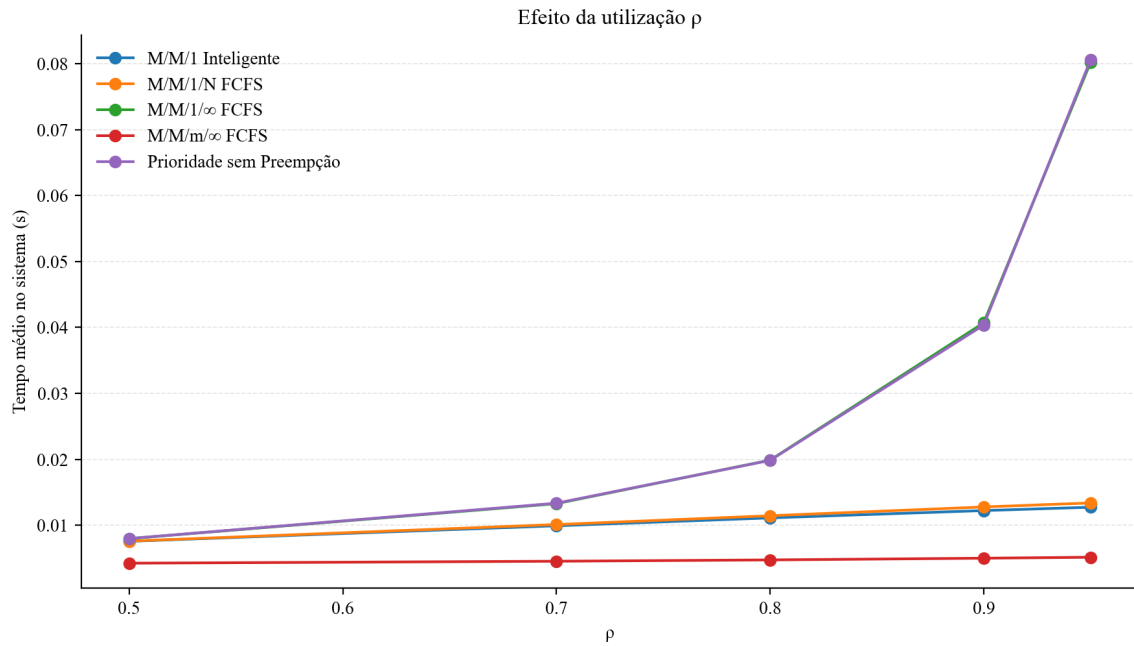


Fig. 3. Efeito da utilização ρ sobre o tempo médio no sistema.

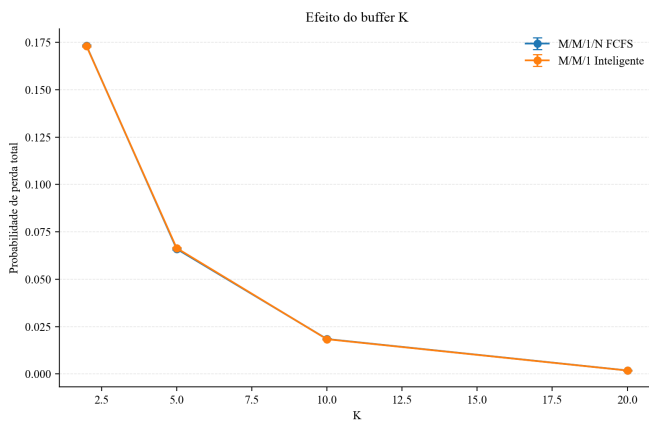


Fig. 4. Efeito do tamanho do buffer sobre a perda total.

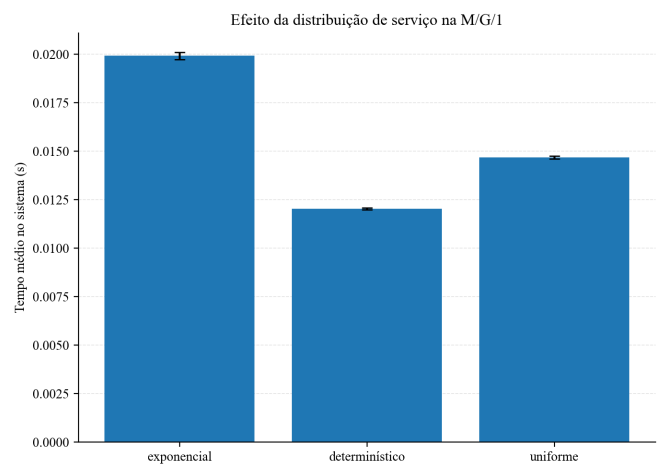


Fig. 5. Efeito da distribuição de serviço na fila M/G/1.

para o fluxo de saídas atendidas. Assim, a validação ajustada é a que melhor representa o balanço do sistema principal.

V. CONCLUSÃO

O estudo confirmou que a fila M/M/1 com buffer inteligente e descarte seletivo atende ao objetivo de proteger tráfego crítico em um roteador IoT com armazenamento limitado. No cenário base, a política manteve a perda total próxima à da M/M/1/N/FCFS, mas reduziu em aproximadamente 98% a perda da classe alta e elevou sua vazão. O custo dessa proteção foi transferido para a classe baixa, que passou a absorver maior parcela dos descartes.

As análises complementares reforçaram resultados esperados da teoria de filas e da literatura recente. O aumento

da utilização intensificou atrasos, buffers maiores reduziram perdas, menor variabilidade de serviço beneficiou a M/G/1 e dois servidores reduziram fortemente o tempo de permanência. Além disso, a validação ajustada da Lei de Little mostrou que a política de substituição foi implementada de forma consistente.

Como próximos passos, a mesma estrutura pode ser estendida para múltiplos níveis de prioridade, limites explícitos de substituição para evitar penalização excessiva da classe baixa, políticas híbridas de descarte antecipado e análise conjunta de atraso, perda e frescor da informação. Outra extensão possível seria usar políticas adaptativas, sensíveis ao histórico recente da fila ou ao atraso estimado dos pacotes, em linha com

propostas recentes de seleção dinâmica de filas por pacote em serviços de baixa latência [7]. Ainda assim, para o escopo deste trabalho, a solução proposta já demonstra vantagem clara quando o requisito principal é preservar pacotes urgentes sem aumentar significativamente a perda total do sistema.

APPENDIX A VISÃO GERAL DOS SCRIPTS

Os códigos completos comentados foram entregues junto ao projeto em arquivos Python. Este apêndice apresenta apenas uma visão geral e trechos representativos das partes centrais da implementação, devido à limitação de espaço do modelo IEEE.

- `common.py`: concentra estruturas e rotinas compartilhadas, incluindo a entidade `Packet`, geração de tempos aleatórios, cálculo de estatísticas, intervalos de confiança, verificação da Lei de Little e escrita dos arquivos CSV e JSON.
- `sim_mml_buffer_inteligente_descarte_seletivo.py`: implementa a fila principal do trabalho, seus cenários de carga, `buffer` e mistura entre prioridades, além da lógica de substituição seletiva no `buffer`.
- `sim_mml_buffer_finito_fcfs.py`: implementa a referência M/M/1/N/FCFS, em que $N = K + 1$ e qualquer chegada é bloqueada quando a capacidade total está esgotada.
- `sim_mml_buffer_infinito_fcfs.py`: implementa a referência M/M/1 com `buffer` infinito, usada como baseline sem perdas por bloqueio.
- `sim_mgl_buffer_infinito_fcfs.py`: implementa a fila M/G/1, mantendo a média do serviço e alternando entre distribuições exponencial, determinística e uniforme.
- `sim_mmm_buffer_infinito_fcfs.py`: implementa a fila M/M/m com dois servidores paralelos, usada para comparar o efeito da capacidade de atendimento.
- `sim_prioridade_sem_preempcao.py`: implementa a fila com duas classes e prioridade sem preempção, na qual a escolha do próximo pacote privilegia a classe alta, mas não interrompe um atendimento já iniciado.
- `make_plots.py`: lê os arquivos `summary.csv` e `class_summary.csv` e produz os gráficos comparativos usados na análise.

APPENDIX B TRECHOS REPRESENTATIVOS

O primeiro trecho representa a definição sistemática dos cenários da fila principal, cobrindo cenário base, variações de utilização, de `buffer` e de probabilidade de tráfego prioritário.

```
return [
    {"scenario_id": "base",
     "lambda_rate": 200.0,
     "mu_rate": 250.0,
     "rho": 0.80, "K": 5,
     "p_high": 0.30},
    {"scenario_id": "rho_0_90",
     "lambda_rate": 225.0,
```

```
     "mu_rate": 250.0,
     "rho": 0.90, "K": 5,
     "p_high": 0.30},
]
```

O segundo trecho sintetiza a transformação de uma replicação em métricas agregadas, incluindo vazão, ocupação média, percentis temporais e validação pela Lei de Little.

```
throughput_total =
    served_total / measured_time
mean_system_occupancy =
    system_occupancy_area / measured_time
mean_system_time =
    mean(system_times)
p95_system_time =
    percentile(system_times, 95.0)
```

O terceiro trecho representa a etapa de pós-processamento que gera os gráficos comparativos a partir dos arquivos resumidos gravados por cada simulador.

```
plot_bar_comparison(
    summary_rows,
    "loss_probability_total",
    "comparacao_perda_total_base.png")
plot_priorityBars(
    class_rows,
    "loss_probability",
    "perda_por_prioridade_base.png")
```

REFERENCES

- [1] C. Blumschein, I. Behnke, L. Thamsen, and O. Kao, "Differentiating network flows for priority-aware scheduling of incoming packets in real-time IoT systems," in *2022 IEEE 25th International Symposium on Real-Time Distributed Computing (ISORC)*, 2022.
- [2] B. Briscoe, K. D. Schepper, M. Bagnulo, and G. White, "Low latency, low loss, and scalable throughput (l4s) internet service: Architecture," RFC Editor, Tech. Rep. RFC 9330, Jan. 2023.
- [3] A. Domański, J. Domańska, T. Czachórski, J. Klamka, J. Szygula, and D. Marek, "The IoT gateway with active queue management," *International Journal of Applied Mathematics and Computer Science*, vol. 31, no. 1, pp. 165–178, Apr. 2021.
- [4] S. T. A. Jafri, I. Ahmed, and S. Ali, "Queue-buffer optimization based on aggressive random early detection in massive NB-IoT MANET for 5g applications," *Electronics*, vol. 11, no. 18, p. Art. no. 2955, Sep. 2022.
- [5] M. P. Abdollahi, H. Azarhava, A. Haghray, and J. M. Niya, "On the rate and age of information for non-preemptive systems with prioritized arrivals and deterministic packet deadlines in IoT networks," *Ad Hoc Networks*, vol. 124, p. 102717, 2022.
- [6] K. D. Schepper, B. Briscoe, and G. White, "Dual-queue coupled active queue management (aqm) for low latency, low loss, and scalable throughput (l4s)," RFC Editor, Tech. Rep. RFC 9332, Jan. 2023.
- [7] S. Ray, X. Jiang, J. Luo, N. Feamster, and J. Jiang, "Swiftqueue: Optimizing low-latency applications with swift packet queuing," arXiv:2410.06112, 2024.